

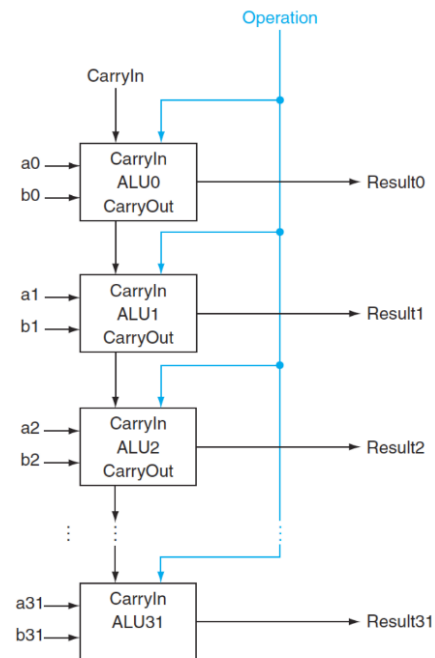
Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (09/19)

1. Consider 32-bit ripple-carry addition shown to the right. Assuming the CarryIn for the least-significant bit's full adder (ALU0) is 0, what is the CarryIn at **bit 18's full adder (ALU18)** when adding $a=0x02C30FD1$ and $b=0x0051EE34$? Show your work.

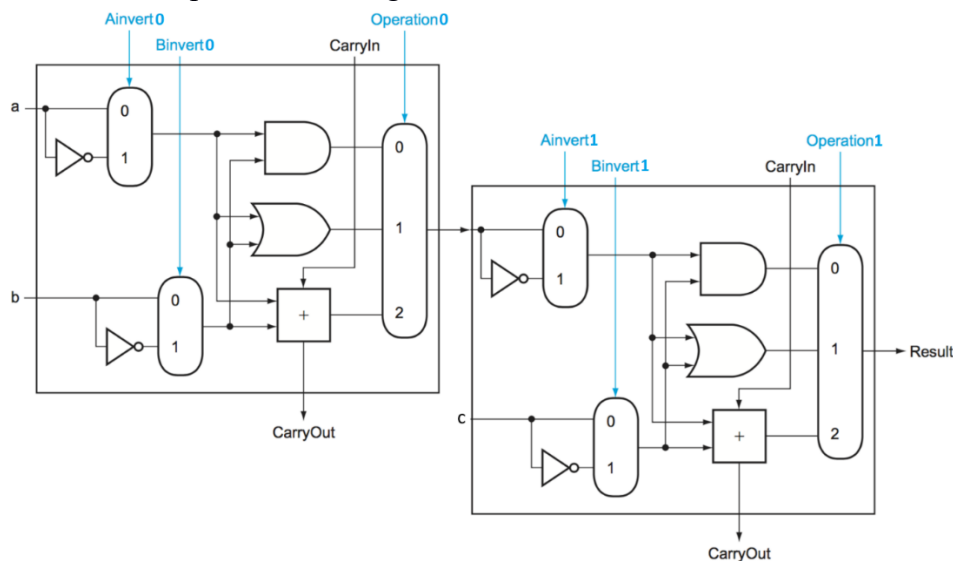
Answer:

1,

0000 0010 1100 0011...
+ 0000 0000 0101 0001...



2. Consider the ALU below with **three 1-bit inputs**: a, b and c. It is constructed from two smaller ALUs. Assume that both of the CarryIn bits are 0. How should the **control signals** (Ainvert0, Binvert0, Operation0, Ainvert1, Binvert1 and Operation1) be set such that the ALU's Result produces the logical **NAND** of a, b and c?



Answers:

Ainvert0	Binvert0	Operation0	Ainvert1	Binvert1	Operation1
1	1	1	0	1	1
0	0	0	1	1	1

...and possibly more

3. The following assembly code shows how to perform **64-bit unsigned integer addition** in our MIPS ISA. The operation performed is $X + Y = Z$, where X, Y and Z are 64-bit unsigned integers each assigned to two 32-bit registers: [$\$s0$] = X[31:0] and [$\$s1$] = X[63:32]; [$\$s2$] = Y[31:0] and [$\$s3$] = Y[63:32]; [$\$t0$] = Z[31:0] and [$\$t1$] = Z[63:32].

```
addu $t0, $s0, $s2
sltu $t7, $t0, $s0
addu $t1, $s1, $s3
addu $t1, $t1, $t7
```

- (a) In this code, which register holds the **carry out** bit from adding the least-significant 32 bits of X and Y?

Answer:
\$t7

- (b) Modify this assembly code to instead perform **64-bit unsigned integer subtraction** such that $X - Y = Z$. Assume that $X \geq Y$.

Answer:
subu \$t0, \$s0, \$s2
sltu \$t7, \$s0, \$t0
subu \$t1, \$s1, \$s3
subu \$t1, \$t1, \$t7